BTK AKADEMİ

İLERİ SEVİYE PYTHON PROGRAMLAMA DİLİ

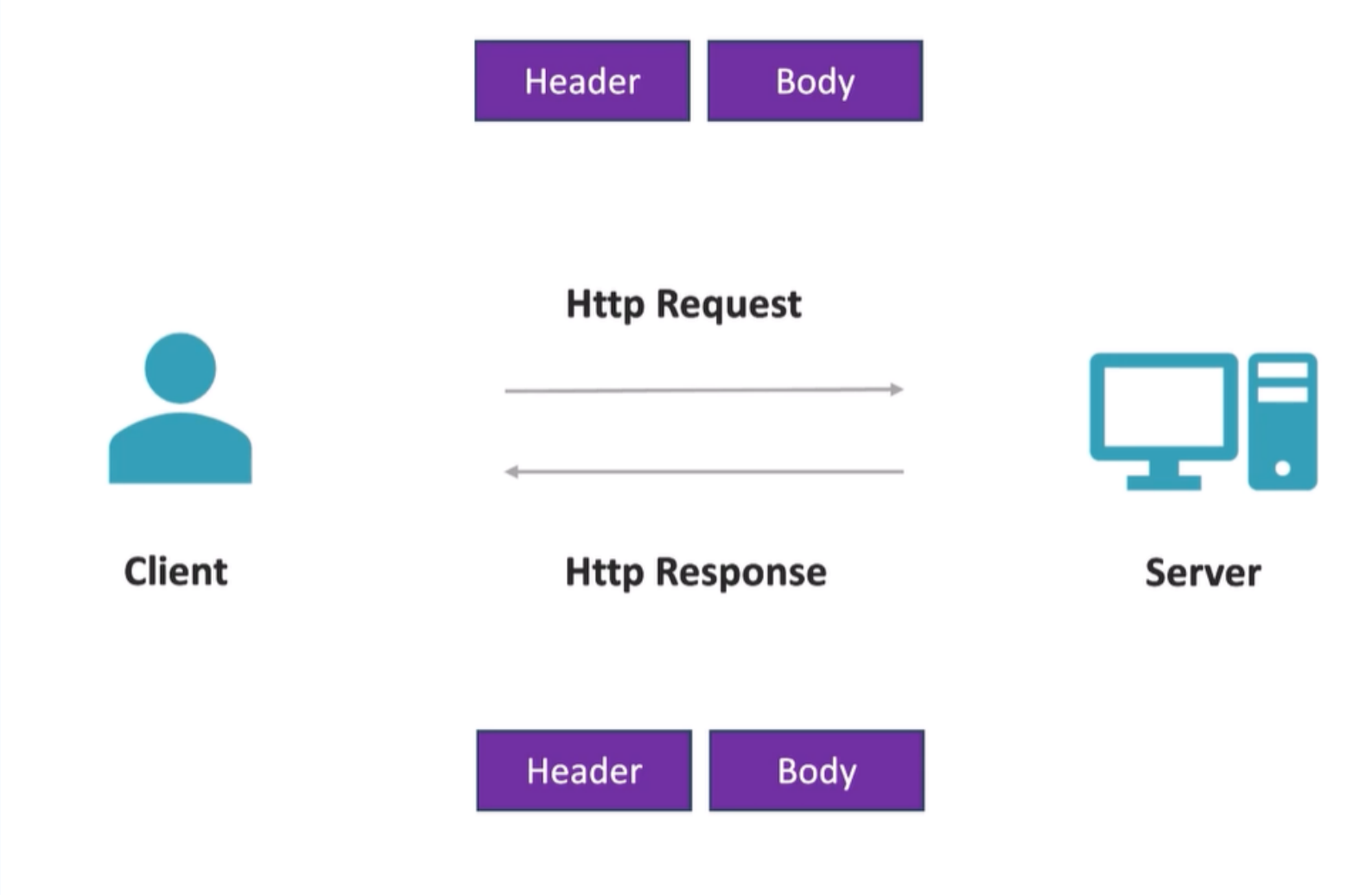
HTTP ve REST API

EBRU AYDOĞAN

İnternet Nasıl Çalışır?

- İnternette yaptığımız her işlem bir **İstek (Request)** ve bir **Yanıt (Response)** üzerine kuruludur.
- **İstemci (Client):** Veriyi talep eden taraftır (Sizin Python kodunuz, tarayıcınız veya mobil uygulamanız).
- **Sunucu (Server):** Veriyi saklayan ve isteğe göre gönderen taraftır.
- Python ile bir web sitesinden veri çektiğimizde, aslında sunucuya "Bana şu veriyi gönder" diyen bir mektup yollamış oluruz.

İnternet Nasıl Çalışır?



BTK AKADEMİ

REST API Nedir?

- **API (Application Programming Interface):** İki farklı yazılımın birbiriyle konuşmasını sağlayan bir arayüzdür.
- **REST (Representational State Transfer):** Web servisleri için en popüler standarttır.
- Bir REST API, sunucudaki kaynaklara (ürünler, kullanıcılar, döviz kurları) standart bir URL üzerinden erişmemizi sağlar.
- **Örnek:** https://api.btk.gov.tr/kurlar adresi bize kurları **JSON** formatında döner.

BTK AKADEMİ

HTTP Metotları

- Sunucuya ne yapmak istediğimizi **Metotlar** ile söyleriz:
  - **GET**: Veri çekmek için kullanılır (En çok bunu kullanacağız).

https://www.btkakademi.gov.tr/

https://www.btkakademi.gov.tr/portal/search?q=python

BTK AKADEMİ

HTTP Metotları

- Sunucuya ne yapmak istediğimizi **Metotlar** ile söyleriz:
  - **POST**: Sunucuya yeni bir veri (bilgi) göndermek için kullanılır.

HTTP Durum Kodları

- Sunucuya ne yapmak istediğimizi **Metotlar** ile söyleriz:
  - **GET:** Veri çekmek için kullanılır (En çok bunu kullanacağız).
  - **POST:** Sunucuya yeni bir veri göndermek için kullanılır.
- Sunucu bize yanıtla birlikte bir **Durum Kodu (Status Code)** gönderir:
  - **200 OK:** Her şey yolunda, veri geldi.
  - **404 Not Found:** Aradığınız adres (endpoint) bulunamadı.
  - **500 Internal Server Error:** Sunucu tarafında bir hata oluştu.

JSON'ın API Dünyasındaki Rolü

- Eskiden veriler XML ile taşınırdı (çok kalabalıktı). Günümüzde REST API'lerin %99'u JSON kullanır.
- Neden?
  - Hafiftir, interneti yormaz.
  - Python sözlük (dict) yapısına doğrudan dönüştürülebilir.
  - Hem insan hem de bilgisayar tarafından okunması çok kolaydır.

BTK AKADEMİ

API Dokümantasyonu Nasıl Okunur?

- Bir API'yi kullanmadan önce onun kılavuzunu (Documentation) okumalıyız.
- **Base URL:** Servisin ana adresi (Örn: https://api.exchangerate.host).
- **Endpoint:** Ulaşmak istediğimiz spesifik kaynak (Örn: /latest).
- **Query Parameters:** Veriyi filtrelemek için sonuna eklediğimiz bilgiler (Örn: ?base=USD).
- **API Key:** Bazı servisler güvenlik için size özel bir anahtar (şifre) ister.

BTK AKADEMİ

requests Modülü ve Kurulumu

- Python'da dış dünya (web siteleri, API'ler) ile iletişim kurmak için en popüler kütüphane **requests** modülüdür.
- Standart kütüphanede yer almaz; "insanlar için HTTP" sloganıyla geliştirilmiş, dışarıdan yüklenen bir pakettir.
- Karmaşık ağ protokollerini, sadece birkaç satır kodla yönetebileceğimiz basit fonksiyonlara indirger.
- **Kurulum (Terminal/CMD):**
  - pip install requests
- Kullanım (Python):
  - import requests # Modülü projemize dahil ediyoruz

BTK AKADEMİ

API'den Veri Çekme (GET Requests)

- **İlk İstek: get() Metodu ve Response Objesi**
  - Bir API'den veri çekmek (okumak) için **requests.get(url)** fonksiyonu kullanılır.
  - Bu fonksiyon sunucuya bir talep gönderir ve karşılığında bir **Response (Yanıt)** objesi döndürür.
  - **Response Objesi:** İçerisinde sadece veriyi değil, işlemin teknik durumunu (başarılı mı, hata mı?) da barındırır.

BTK AKADEMİ

API'den Veri Çekme (GET Requests)

- Kod Örneği:

```
import requests

# Test amaçlı bir API adresine GET isteği atıyoruz
response = requests.get("https://jsonplaceholder.typicode.com/posts")

# Yanıtın tipini kontrol edelim
print(type(response)) # <class 'requests.models.Response'>
```

BTK AKADEMİ

API'den Veri Çekme (GET Requests)

- Yanıtın İçeriğine Erişmek (Status Code & Text)
  - Sunucudan dönen yanıtın detaylarını şu özelliklerle inceleriz:
    - **.status_code:** İsteğin sonucunu sayısal kodla verir (Örn: 200 = Başarılı, 404 = Bulunamadı).
    - **.text:** Gelen veriyi ham metin (string) olarak verir. Genellikle JSON formatında bir metindir.
    - **.encoding:** Gelen metnin karakter setini (Örn: utf-8) gösterir.

API'den Veri Çekme (GET Requests)

- Kod Örneği:

```python
import requests

response = requests.get("https://jsonplaceholder.typicode.com/posts")

# Bağlantı durumunu kontrol edelim
print(f"Durum Kodu: {response.status_code}") # 200

if response.status_code == 200:
    # İlk 100 karakteri görelim
    print(response.text[:100])
```

BTK AKADEMİ

JSON Verisini Python Objesine Çevirme

- .text ile gelen veri bir string'dir ve üzerinde işlem yapmak zordur.
- **.json()** metodu, bu string'i otomatik olarak Python'un anlayacağı **Liste (List)** veya **Sözlük (Dictionary)** yapılarına dönüştürür.
- Bu aşamadan sonra veriye tıpkı sabah oturumunda öğrendiğimiz JSON dersindeki gibi erişebiliriz.

JSON Verisini Python Objesine Çevirme

- Kod Örneği:

```python
import requests

response = requests.get("https://jsonplaceholder.typicode.com/posts")

# Ham metni Python listesine çeviriyoruz
posts = response.json()

# Artık standart liste işlemleri yapabiliriz
for post in posts[:2]: # Sadece ilk iki gönderiyi yazdıralım
    print(f"ID: {post['id']} | Başlık: {post['title']}")
    print("-" * 20)
```

BTK AKADEMİ

Query Parameters (Filtrelenmiş Kayıt Çekme)

- Bazen API'deki tüm veriyi değil, sadece belirli kriterlere uyanları çekmek isteriz.
- Bunun için URL sonuna parametre eklemek yerine, get() fonksiyonuna **params** sözlüğü göndermek en güvenli yoldur.
- Bu sayede Python, URL içindeki özel karakterleri (boşluk, &, ?) bizim yerimize hatasız şekilde düzenler.

Query Parameters (Filtrelenmiş Kayıt Çekme)

- Kod Örneği:

```python
import requests

# Ana adres (Base URL)
url = "https://jsonplaceholder.typicode.com/posts"

# Filtreleme kriterlerini sözlük olarak tanımlıyoruz
# Örn: Sadece userId: 1 ve id: 5 olan kaydı getir
query_params = {
    "userId": 1,
    "id": 5
}

# params parametresi ile güvenli gönderim
response = requests.get(url, params=query_params)

# Yanıtı doğrudan Python listesine çeviriyoruz
result = response.json()
print(result) # Çıktı: [{'userId': 1, 'id': 5, ...}]
```

BTK AKADEMİ

Dinamik Filtreleme ve Kullanıcı Etkileşimi

- Statik değerler yerine, kullanıcıdan alınan verileri API sorgularına dahil ederek **interaktif uygulamalar** geliştirebiliriz.
- Bu yapı, arama motorları veya filtreleme sistemlerinin temel çalışma mantığıdır.
- Kullanıcıdan input() ile alınan veri, params sözlüğü üzerinden doğrudan API'ye iletilir.

Dinamik Filtreleme ve Kullanıcı Etkileşimi

- Kod Örneği:

```python
import requests

base_url = "https://jsonplaceholder.typicode.com/comments"

# Kullanıcıdan filtreleme kriteri alalım
post_id = input("Hangi gönderinin yorumlarını görmek istersiniz? (ID giriniz): ")

# Dinamik sözlük yapısı
payload = {"postId": post_id}

# API isteği
response = requests.get(base_url, params=payload)

if response.status_code == 200:
    comments = response.json()
    print(f"\n{post_id} numaralı gönderi için {len(comments)} yorum bulundu:")
    for c in comments[:2]: # Sadece ilk 2 yorumu gösterelim
        print(f"- {c['email']}: {c['name']}")
```

BTK AKADEMİ

Sunucuya Veri Göndermek (POST Requests)

- **GET** veriyi okumak için kullanılırken; **POST**, sunucuda yeni bir kaynak oluşturmak (Örn: Yeni kullanıcı kaydı, tweet atma, form gönderme) için kullanılır.
- POST isteği atarken gönderilecek veri, URL'nin sonunda değil, isteğin **gövdesinde (body)** gizli olarak gider.
- Sunucu işlemi başarıyla tamamlarsa genellikle **201 Created** durum kodunu döner.
- **Temel Parametreler:**
  - url: İşlemin yapılacağı adres.
  - data: Gönderilecek veri (Genellikle sözlük yapısında).
  - headers: Sunucuya verinin tipini (Örn: JSON) bildiren başlık bilgileri.

- Kod Örneği:

```python
import requests

url = "https://jsonplaceholder.typicode.com/posts"

# Göndermek istediğimiz veri (Sözlük yapısı)
new_post = {
    "title": "BTK Akademi İleri Python",
    "body": "Bu içerik POST isteği ile oluşturuldu.",
    "userId": 1
}

# Verinin tipini sunucuya bildiriyoruz (Header)
headers = {"Content-type": "application/json; charset=UTF-8"}

# POST isteğini gerçekleştiriyoruz
response = requests.post(url, json=new_post, headers=headers)

# Sonuçları inceleyelim
if response.status_code == 201:
    print("Başarıyla Oluşturuldu!")
    # Sunucudan dönen yeni kayıt bilgisini görelim
    print(response.json())
else:
    print(f"Hata! Durum Kodu: {response.status_code}")
```

BTK AKADEMİ

Uygulama 1 – Canlı Hava Durumu Hattı

- **Senaryo:** Bir API servisinden (WeatherApi vb.) şehir ismine göre canlı hava durumu verilerini çekmek, bu verileri Regex ile filtrelemek ve günün sonunda bir rapora dönüştürmek.
- **İşlem Adımları:**
  - Kullanıcıdan şehir ismi al.
  - API'ye GET isteği gönder.
  - Gelen karmaşık JSON içinden "Derece" ve "Durum" bilgisini ayıkla.
- url: http://api.weatherapi.com/v1/current.json
- Api key: Üyelik sonrası üretilen key kullanılmalı.

BTK AKADEMİ